

MCP-Architektur-Entwurfsfragen für Enterprise-Deployments (Stand: September 2026)

TL;DR

Die technische Basisschicht von MCP ist heute praxisreif und offiziell standardisiert (OAuth-2.1-Autorisierung mit strikter Resource-Server/Authorization-Server-Trennung seit der Spec-Revision 2025-06-18, Tool-Annotationen seit 2025-03-26, Elicitation/Sampling released in 2025-11-25); die *Governance- und Betriebsschicht* (Audit-Logging, Tool-Discovery bei Skalierung, Token-Delegation an Backends, Human-Approval) ist dagegen nur teilweise standardisiert und muss weitgehend selbst gebaut oder über Gateways zugekauft werden.

Für die Enterprise-Kernentscheidungen empfiehlt sich: OAuth 2.1 mit Token-Exchange (RFC 8693) statt Token-Passthrough (von der Spec explizit als Anti-Pattern per MUST NOT verboten), konsolidierte, aufgabenorientierte Tools statt API-1:1-Wrapper, harte Read/Write-Trennung mit Human-in-the-Loop-Approval für schreibende/destruktive Operationen, und ein Gateway/Orchestrator (Muster C) als Kontrollebene, sobald mehr als ein Handvoll Backend-Server im Spiel ist.

Für das konkrete Szenario (LLM-Chat-Produkt an Backend mit strukturierten Daten + Vector Search + Graph Search) ist die stärkste Architektur ein **eigener, vorgelagerter MCP-Server (Muster B) hinter einem leichten Gateway/Orchestrator (Muster C)**; Muster A (nativer Plattform-MCP-Server) nur nutzen, wenn der Anbieter einen produktionsreifen, sicherheitsgeprüften nativen Server bereitstellt.

Key Findings

Auth ist standardisiert, Delegation ist es nicht. OAuth 2.1 ist offizieller Spec-Standard: Der MCP-Server ist ausschließlich *Resource Server*, ein separater *Authorization Server* stellt Tokens aus. Token-Passthrough (unverändertes Weiterreichen des eingehenden Tokens an Backends) ist per **MUST NOT** verboten. Wie aber die Endnutzer-Identität sauber an nachgelagerte APIs propagiert wird (Token-Exchange nach RFC 8693), ist *nicht* Teil der MCP-Spec und muss selbst gelöst werden.

Tool-Overload ist das dominierende Skalierungsproblem. Empirisch belegt: zu viele Tool-Definitionen im Kontext senken die Tool-Auswahl-Genauigkeit deutlich und erhöhen Kosten/Latenz. Die Standardarchitektur (alle Tools per `tools/list` upfront laden) skaliert nicht; State-of-the-Art-Gegenmittel sind Tool-Filtering, retrieval-basierte Tool-Auswahl (RAG-MCP), Namespacing und dynamisches Nachladen.

Read/Write-Trennung ist die wichtigste Sicherheits-Architekturentscheidung. Die Spec liefert Annotationen (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`), diese sind aber nur *Hints*, keine erzwingbaren Kontrakte – Enforcement muss deterministisch im Server/Gateway erfolgen.

Elicitation und Sampling sind jetzt released (Revision 2025-11-25), aber Client-Support ist uneinheitlich; Human-Approval ist in der Praxis überwiegend außerhalb der Spec (im Host/Client oder Gateway) umgesetzt.

Native MCP-Logging-Capability ist nur ein Debug-Utility – Compliance-taugliche Audit-Trails muss man selbst bauen oder über ein Gateway beziehen.

MCP-Gateways sind der de-facto Enterprise-Standard für Auth-Zentralisierung, Audit, Rate-Limiting und Tool-Aggregation; das Feld ist jung, teils Marketing-getrieben, und Feature-Sets variieren stark.

Details

1. Grundprimitive – nur die architekturelevante Vertiefung

MCP kennt server-seitige Primitive (**Tools, Resources, Prompts**) und client-seitige Primitive (**Roots, Sampling, Elicitation**). Für Architekturentscheidungen zählt vor allem die *Kontrollhoheit*:

Tools = model-controlled. Das LLM entscheidet autonom, *ob* und *wann* ein Tool aufgerufen wird. Damit tragen Tools das höchste Risiko und sind der Fokus aller Security-Kontrollen (Auth, Approval, Read/Write-Trennung). Alles, was Seiteneffekte erzeugt oder schreibt, gehört als Tool modelliert – mit entsprechenden Annotationen und Guardrails.

Resources = application-controlled. Die Host-Anwendung entscheidet, welche Daten (Dateien, DB-Auszüge, Schemas) in den Kontext geladen werden. Resources sind ideal für *lesenden, kuratierten* Kontext, den man deterministisch steuern will, statt das Modell frei „stöbern“ zu lassen. In der Praxis eignen sie sich z.B. für Schema-Beschreibungen einer Datenplattform.

Prompts = user-controlled. Explizit vom Nutzer ausgelöste Templates/Workflows (z.B. Slash-Commands). Gut für wiederkehrende, geführte Abläufe.

Architekturkonsequenz: In der Praxis wird fast alles als *Tool* gebaut, weil die meisten Clients (v.a. Coding-Agents) Resources und Prompts schwächer unterstützen. Das ist ein pragmatischer Kompromiss, verschiebt aber Kontrolle vom Application-Layer ins Modell – ein Grund mehr, Read/Write-Trennung und Approval sauber zu lösen. **Empfehlung:** Read-only, kuratierten Kontext (z.B. Schema-/Dokumentbestände) wo möglich als Resources modellieren, um die Anzahl model-controlled Tools zu reduzieren.

2. Authentifizierung und Autorisierung

OAuth 2.1 als Spec-Standard. Die Autorisierung ist im MCP-Spec-Kapitel „Authorization“ definiert (verankert u.a. in den Revisionen 2025-06-18, 2025-11-25 und 2026-07-28 auf modelcontextprotocol.io). Kernpunkte (verifiziert an der offiziellen Spec, Revision 2025-11-25):

Autorisierung ist **OPTIONAL**, gilt aber für HTTP-Transporte (STDIO-Server sollen **NICHT** dem OAuth-Flow folgen, sondern Credentials aus der Umgebung ziehen).

Der MCP-Server agiert **ausschließlich als OAuth 2.1 Resource Server**; er validiert Tokens, verwaltet aber keine Logins/Token-Ausgabe. Der **Authorization Server** ist eine separate Instanz (kann Auth0, Okta, Keycloak, Entra ID etc. sein). Diese Trennung wurde mit der Revision vom Juni 2025 verbindlich.

MCP-Server **MÜSSEN** OAuth 2.0 Protected Resource Metadata (RFC 9728) implementieren; Clients **MÜSSEN** PKCE nutzen; Clients **MÜSSEN** den `resource`-Parameter (RFC 8707 Resource Indicators) in Authorization- und Token-Request senden. Letzteres verhindert, dass ein für Server A ausgestelltes Token bei Server B funktioniert (Audience-Bindung).

Der Server **MUSS** validieren, dass Tokens explizit für ihn (Audience) ausgestellt wurden.

Token-Passthrough = verbotenes Anti-Pattern (verifiziert). Die offizielle „Security Best Practices“-Seite der Spec benennt Token-Passthrough explizit: „*Token passthrough*“ is an anti-pattern where an MCP server accepts tokens from an MCP client without validating that the tokens were properly issued to the MCP server and passes them through to the downstream API. Es ist per **MUST NOT** verboten, weil es (a) Rate-

Limiting/Audience-Kontrollen umgeht, (b) das *Confused-Deputy*-Problem erzeugt, (c) Trust-Boundaries zwischen Services bricht und (d) Auditing/Attribution zerstört.

Der korrekte Weg zum Backend: Token-Exchange (RFC 8693). Statt das eingehende Nutzer-Token durchzureichen, tauscht der MCP-Server (oder das Gateway) es gegen ein *scoped, downstream-spezifisches* Token. Dieser Mechanismus ist jedoch **nicht Teil der MCP-Spec** – er wird über den Identity Provider gelöst (z.B. Workforce/Workload Federation). Das ist eine der wichtigsten „selbst zu bauenden“ Lücken.

Zwei Autorisierungsebenen sauber trennen:

Protokoll-Ebene (Client ↔ Server): „Darf dieser Client überhaupt mit diesem Server reden?“ → OAuth 2.1 Token-Validierung.

Tool-/Aktions-Ebene: „Darf dieser Nutzer *dieses Tool* mit *diesen Parametern* ausführen?“ → RBAC/ABAC im Server bzw. Gateway, nicht vom Protokoll abgedeckt. Least-Privilege gilt hier: kurzlebige, minimal-scoped Tokens; ein datenlesender Server soll keine Schreibrechte haben.

Enterprise-Identitätspropagation (LLM-Host → MCP-Client → MCP-Server → Backend): Best Practice ist, die *echte* Endnutzer-Identität durchzureichen (delegierte Nutzerauthentifizierung) statt eines geteilten Service-Accounts. Ein einzelner Service-/Admin-Account als Backend-Credential ist ein Anti-Pattern (hoher Blast-Radius, umgeht Backend-RBAC, zerstört Attribution). Realistischer Enterprise-Pattern: Nutzer authentifiziert sich beim Authorization Server → Client erhält scoped Token für den MCP-Server → Server tauscht (RFC 8693) gegen ein Backend-Token, das die Nutzeridentität und minimale Scopes trägt. Service-Accounts nur für nicht-nutzerbezogene System-Operationen.

3. Tool Discovery

Technische Funktionsweise: Der Client ruft `tools/list` auf und erhält Name, Beschreibung und JSON-Schema (`inputSchema`, optional `outputSchema`) je Tool. Ausführung via `tools/call`. Bei dynamischen Änderungen sendet der Server `notifications/tools/list_changed`; Clients sollten die Liste dann neu laden. `*/list`-Methoden sind per opakem Cursor paginiert.

Statisch vs. dynamisch: Statische Discovery (feste Toolmenge) ist der Normalfall und robust. Dynamische Discovery (`list_changed`) ist Spec-konform, aber sicherheitssensibel: ein zur Laufzeit „gewechseltes“ Tool (Rug-Pull) ist ein bekanntes Angriffsmuster – Änderungen an Beschreibungen/Annotationen sollten überwacht und ggf. neu freigegeben werden.

Kernproblem: Kontext-Overload. Empirisch gut belegt: mehr Tools ≠ besser. Jede Tool-Definition kostet Kontext-Token; ab einer bestimmten Menge sinkt die Auswahl-Genauigkeit spürbar und Latenz/Kosten steigen. Belege:

Cursor deckelt hart bei 40 Tools; Windsurf bei 100.

Der offizielle GitHub-MCP-Server verbraucht ~42.000 Token allein für Tool-Definitionen: Laut der *getunblocked*-Messung „GitHub MCP Token Cost: A 2026 Autopsy“ gilt: „If you have the official GitHub MCP server installed in Claude Code, roughly 42,000 tokens of your context window are spent before you type your first prompt.“ Eine neuere Zählung (Piotr Hajdas) kommt auf 55.000 Token über 93 Tool-Definitionen – rund 21% eines 200K-Kontextfensters.

Der DEV-Community-Bericht „MCP Tool Overload: Why More Tools Make Your Agent Worse“ misst: „accuracy on correct tool selection dropped from ~95% with a focused toolset to ~71% with the full GitHub MCP server loaded. That's a 24-point accuracy gap caused purely by context bloat.“

Akademische Benchmarks (MTU-Bench, TaskBench) zeigen konsistent: mit steigender Toolzahl fällt die Genauigkeit; TaskBench meldet z.B. einen Abfall der Graph-Accuracy von ~96% (1 Tool) auf ~25% (8 Tools). [NeurIPS](#)

Anthropic bestätigt in „Code execution with MCP“ (04.11.2025) verbatim: „Today developers routinely build agents with access to hundreds or thousands of tools across dozens of MCP servers. However, as the number of connected tools grows, loading all tool definitions upfront and passing intermediate results through the context window slows down agents and increases costs.“

Beschreibungsqualität als kritischer Erfolgsfaktor: Detailliertere, klar abgegrenzte Beschreibungen verbessern die Auswahl messbar (MetaTool-Benchmark, Anthropic). Anthropic konnte allein durch präzisere Tool-Beschreibungen State-of-the-Art auf SWE-bench Verified mit Claude 3.5 Sonnet erreichen (qualitativ berichtet, ohne konkrete Punktzahl im Blogtext).

Namenskollisionen: MCP hat einen *flachen* Namespace pro Server; Tool-Namen sind nur *innerhalb eines Servers* eindeutig. Beim Aggregieren mehrerer Server drohen Kollisionen. Microsoft Research („Tool-space interference in the MCP era“, Analyse von 1.470 MCP-Servern) fand verbatim: „we found name collisions between 775 tools. The most common collision was 'search', which appears across 32 distinct MCP servers.“ Die Spec (Revision 2026-07-28) sagt: Clients/Proxies, die aggregieren, **SOLLTEN** disambiguieren, z.B. durch Präfixe (`[asana_search]`, `[jira_search]`). Clients gehen unterschiedlich damit um (Cursor: `[mcp_<server>_<tool>]`; manche „last-registered wins“ – ein Sicherheitsrisiko, da ein bössartiger Server einen legitimen überschreiben kann).

Stand-der-Technik-Lösungsansätze (mit Reifegrad-Einordnung):

Tool-Filtering / statisches Deaktivieren (reif, trivial): nur benötigte Tools aktivieren.

Namspacing/Präfixe (reif, Community-Best-Practice, in Spec als SHOULD): löst syntaktische, nicht semantische Kollisionen.

Retrieval-basierte Tool-Auswahl (RAG-MCP / „Wrapper Pattern“) (aufkommend, teils experimentell): statt alle Tools zu laden, wird per Vektor-Suche der passende Toolset dynamisch selektiert. Redis berichtet 98% weniger Token-Nutzung und verdoppelte Genauigkeit; das RAG-MCP-Paper meldet >90% Erfolg bei <30 Tools und deutliche Degradation jenseits ~100 Tools. Diese Zahlen stammen von interessierten Anbietern/Autoren und sollten als indikativ, nicht als neutral validiert gelten. [GitHub](#)

Code-Execution / „Progressive Disclosure“ (neu, von Anthropic propagiert, 04.11.2025): Tools werden als Code-API präsentiert; der Agent lädt Definitionen on-demand vom „Filesystem“ oder via `[search_tools]`. Anthropic verbatim: „This reduces the token usage from 150,000 tokens to 2,000 tokens—a time and cost saving of 98.7%.“ Vielversprechend, aber jung und setzt eine Code-Execution-Umgebung voraus. [anthropic](#)

↗

4. Read-Tools vs. Write-Tools

Warum architektonisch zentral: Lesende Operationen sind i.d.R. reversibel und niedrigrisiko; schreibende/destruktive Operationen (Löschen, Überschreiben, Transaktionen) sind potenziell irreversibel und hochriskant – besonders wenn ein nicht-deterministischer Agent sie durch Prompt-Injection getriggert ausführt. Die Trennung ermöglicht unterschiedliche Auth-Scopes, Approval-Gates und Audit-Tiefe.

Spec-Stand (Tool-Annotationen): Eingeführt in Revision 2025-03-26 (PR #185, Basil Hosmer/Anthropic).

Fünf Felder in `ToolAnnotations`:

`title` (nur UX),

`readOnlyHint` (Tool liest nur; Default `false`),

`destructiveHint` (kann zerstörerisch überschreiben/löschen; nur relevant wenn `readOnlyHint=false`;

Default `true`),

`idempotentHint` (wiederholter Aufruf ohne Zusatzeffekt; Default `false`),

`openWorldHint` (interagiert mit externer Welt/Internet).

Kritische Einschränkung: Annotationen sind **Hints, keine erzwingbaren Kontrakte**. Ein

bösartiger/fehlerhafter Server kann ein destruktives Tool als `readOnlyHint:true` deklarieren, um

Bestätigungsdialoge zu umgehen. Die Spec (Revision 2026-07-28) sagt daher: Clients **MÜSSEN**

Annotationen von nicht-vertrauenswürdigen Servern als *untrusted* behandeln. Zudem deklarieren nur sehr wenige reale Tools überhaupt Annotationen – ein schreibendes Tool sieht „auf dem Draht“ oft identisch zu einem lesenden aus.

Praktische Patterns:

Getrennte Server für Read vs. Write (z.B. ein reiner Query-Server ohne Schreibrechte, ein separater Write-Server mit strengerem Auth-Scope und Approval). Klarste Trust-Boundary, empfohlen für hochregulierte Umgebungen.

Annotation-basierte Unterscheidung innerhalb eines Servers – bequemer, aber nur so sicher wie das deterministische Enforcement dahinter.

Gateway-Mapping: Manche Proxies (z.B. MCPProxy „DeriveCallWith“) mappen Annotationen auf getrennte Berechtigungsmodi (`call_tool_read`/`_write`/`_destructive`) und flaggen Annotation-Drift.

Write nur mit Bestätigung: Enterprise-Deployments gaten Write-/destruktive Operationen typischerweise mit Human-Approval (siehe §8).

Empfehlung: Read/Write auf *zwei Ebenen* trennen – (1) Annotationen setzen (für UX/Client), (2) das eigentliche Enforcement deterministisch im Server/Gateway per Scope + Approval, *nie* nur auf Hints verlassen.

5. Error Handling

Zwei-Ebenen-Modell (Spec):

Protokoll-Fehler → JSON-RPC-2.0-Error-Objekte (z.B. `-32601` Method not found, `-32602` Invalid params, `-32603` Internal error, `-32001` Timeout). Diese landen beim Client, werden ggf. im UI angezeigt und *nicht* zwingend an das LLM zurückgespielt.

Tool-Ausführungsfehler → **erfolgreiche** JSON-RPC-Response mit `isError: true` im Result und beschreibendem Text im `content`. Die Spec (Tools-Kapitel) sagt: bei fehlgeschlagener Ausführung **SOLLTE** der Server `isError` auf `true` setzen und Fehlerdetails im Content liefern – Fehlerbehandlung gehört in das *Tool-Ergebnis*, nicht auf die Protokoll-Ebene.

Warum das für LLMs entscheidend ist: Tool-Errors mit `isError:true` werden – wie erfolgreiche Antworten – in den Kontext des LLM injiziert, sodass es *selbst korrigieren* kann (falschen Parameter neu setzen, anderes Tool wählen). Protokoll-Fehler dagegen werden oft verworfen und führen zu generischem „Tool execution failed“, worauf das Modell nicht sinnvoll reagieren kann → Halluzinationsgefahr. Häufiger

Implementierungsfehler: Exceptions als Protokoll-Fehler (-32603) durchzureichen statt als `isError`-Result (dokumentiert u.a. in ash_ai #158, ruby-sdk #159).

Praktische Regeln:

Fehlermeldungen als *handlungsleitende* Klartext-Hinweise formulieren (nicht Stacktrace), z.B. „Instanz läuft noch, zuerst `stop_instance` aufrufen.“

Keine sensiblen Daten in Fehlern leaken.

Retry-Strategie *nach Fehlertyp*: nur transiente/Protokoll-Fehler mit Exponential-Backoff wiederholen; Business-Logik-Fehler dem LLM überlassen. Circuit-Breaker/Bulkhead für instabile Backends.

Timeouts: Default oft 60s (`MCP_SERVER_REQUEST_TIMEOUT`); besser: explizite kürzere Timeouts auf Backend-Calls + Progress-Notifications + Pagination statt Timeout-Erhöhung.

Verkettete Tool-Calls (Tool 3 von 5 schlägt fehl): MCP hat *kein* natives Transaktions-/Saga-Konzept über mehrere Tool-Calls. Der Agent-Loop entscheidet, ob er abbricht, kompensiert oder alternativ weitermacht – das ist Sache der Orchestrierung im Host/Framework, nicht des Protokolls. Für schreibende Ketten empfiehlt sich: idempotente Tools (`idempotentHint`), explizite Kompensations-Tools, und Approval-Gates vor irreversiblen Schritten.

6. Logging und Auditability

Native MCP-Logging-Capability = Debug-Utility, nicht Compliance. Die Spec kennt eine optionale `logging`-Capability (strukturierte Log-Messages via JSON-RPC), aber sie ist ausdrücklich für temporäres Debugging gedacht: keine Vorgaben zu Persistenz, zentraler Sammlung oder Identitäts-Threading. JSON-RPC ist stateless und bindet Logs an temporäre Session-IDs – es gibt keinen nativen Standard, um Aktionen rückwirkend einer persistenten Nutzeridentität zuzuordnen.

Was für Compliance geloggt werden muss: timestamp, Principal (Nutzer *und* Agent-Identität), Tool-Name + Server, übergebene Argumente, zurückgegebenes Ergebnis, Latenz, Auth-Kontext/Token-Referenz, Approval-Entscheidung (wer/wann). Auditoren erwarten strukturierte, abfragbare Felder (timestamp, principal, action, resource, outcome) – genau das liefert MCP nativ *nicht*, jede Deployment „erfindet es neu oder lässt es weg“.

Besonderheit KI-Agent-Attribution: Für regulierte Umgebungen (SOC 2, HIPAA, GDPR, ISO 27001, SOX/PCI-DSS) muss nachvollziehbar sein, *was ein Agent im Namen welches Nutzers* getan hat – besonders bei schreibenden Operationen. Das NIST AI RMF nennt Traceability/Accountability als Kernfunktion. Der Gravitee-Report „State of AI Agent Security 2026“ (April 2026, 919 Befragte) berichtet verbatim: „An overwhelming 88% of organizations report either confirmed or suspected AI agent security or privacy incidents within the last year“ (im Gesundheitswesen 92,7%; nur 21,9% behandeln Agents als eigenständige Identitäten). Audit-Trails sind die Voraussetzung für Incident-Response.

Praktische Umsetzung: Der natürliche Ort für Audit ist ein **Gateway/Proxy**, durch das jeder Tool-Call fließt – dort werden OpenTelemetry-Traces, Prometheus-Metriken und strukturierte, unveränderliche Audit-Logs (SIEM-exportierbar) erzeugt. Wichtig: Content-Logging (Argumente/Ergebnisse können PII enthalten) muss pro Umgebung abschaltbar sein, ohne den restlichen Trail zu verlieren. STDIO-Server erzeugen keinen Netzwerkverkehr → dort muss Telemetrie in die Runtime eingebettet werden.

Tools/Frameworks (kritisch, nicht als Endorsement): Bifrost (Maxim AI, Open Source, wirbt mit ~11µs Overhead), MintMCP, Tetrade (OpenInference), Ithena (`ithena-cli` wrappt STDIO-Server), Aembit. Diese

sind Anbieter-Quellen; Feature-Aussagen kritisch prüfen.

7. Tool-Granularität

Trade-off: Wenige, mächtige Tools reduzieren Kontext-Overload und Auswahl-Fehler, verlangen aber komplexere Input-Schemata; viele feinkörnige Tools sind einfacher je Tool, überladen aber den Kontext und erhöhen Kollisions-/Verwechslungsrisiko.

Empirie zur Toolzahl: Siehe §3 – konsistenter Genauigkeitsabfall mit steigender Toolzahl (MTU-Bench, TaskBench, AppSelectBench; Praxis-Deckel 40–100 Tools bei Cursor/Windsurf). Es gibt keine harte „magische Zahl“, aber die Faustregeln aus der Praxis: Toolset klein halten (grob einstellige bis niedrige zweistellige Zahl aktiver Tools pro Agent), Tool-Definitionen möglichst unter ~40% des Kontextfensters halten (Eclipse-Source-Faustregel).

Anthropics offizielle Granularitäts-Empfehlung („Writing effective tools for agents“, 11.09.2025) ist die belastbarste Quelle:

Nicht API-Endpunkte 1:1 als Tools wrappen: „More tools don't always lead to better outcomes. A common error we've observed is tools that merely wrap existing software functionality or API endpoints.“

Konsolidieren auf wenige, hochwirksame, aufgabenorientierte Tools: statt

`[list_users] + [list_events] + [create_event]` ein `[schedule_event]`; statt `[read_logs]` ein `[search_logs]`; statt `[get_customer_by_id] + [list_transactions] + [list_notes]` ein `[get_customer_context]`. [anthropic](#)

Search-fokussiert statt list-all: [search_contacts]/[message_contact] statt [list_contacts] – sonst „verschwendet“ der Agent Kontext mit Brute-Force-Durchlesen. [anthropic](#)

Namespacing per Präfix ([asana_search]) – Prefix- vs. Suffix-Namespacing hat laut Anthropic „non-trivial effects on our tool-use evaluations“.

Sinnvollen Kontext zurückgeben, keine technischen IDs: lieber [name]/[image_url]/[file_type] als [uuid]/[mime_type]; das Auflösen kryptischer UUIDs in semantische Namen „significantly improves Claude's precision ... by reducing hallucinations“.

Response-Format-Parameter ([concise]/[detailed]) anbieten; Claude Code deckelt Tool-Responses per Default auf 25.000 Token; ein konkretes Beispiel spart mit „concise“ ~2/3 der Token (72 statt 206).

[anthropic](#)

8. Human Approval / Human-in-the-Loop

Elicitation (released in 2025-11-25, zuvor Draft). Server können via `[elicitation/create]` strukturierte Nutzereingaben *mitten in* einer Tool-Ausführung anfordern; der Client rendert Formular/Dialog. Drei Antwortzustände: **accept / decline / cancel**. Schema muss flach sein (nur primitive Felder). Wichtige Sicherheitsregel: Elicitation **DARF NICHT** (Form-Mode) für sensible Daten (Passwörter, API-Keys, Tokens, Zahlungsdaten) genutzt werden – dafür gibt es den **URL-Mode**. Elicitation ist der spec-native Baustein für Approval-/Confirm-Dialoge, aber Framework-/Client-Support ist noch begrenzt.

Sampling (released in 2025-11-25). Der Server kann via `[sampling/createMessage]` eine LLM-Completion *durch den Client* anfragen (der Client hält den Modellzugang/Keys). Menschliche Kontrolle ist eingebaut: der Client kann dem Nutzer Prompt *und* Completion zur Prüfung/Bearbeitung/Ablehnung vorlegen (zwei optionale Human-Checkpoints). Sampling ist primär für *server-seitige Agentik* (Zusammenfassen,

Klassifizieren, Fehleranalyse) gedacht, nicht per se für Approval – kann aber Approval-Vorschläge generieren. Hinweis: Nicht alle Clients unterstützen Sampling (z.B. Claude Desktop lange nicht).

Wichtige Klarstellung: In der Praxis wird Human-Approval für Write-/destruktive Tool-Calls heute *überwiegend außerhalb der MCP-Primitive* umgesetzt – im Host/Client (Bestätigungsdialog vor `tools/call`) oder im Gateway (z.B. „manual approval mode“, MCP Guardian, Port, Bifrost). Das ist robuster als sich auf Elicitation-Support zu verlassen.

Enterprise-Patterns (aus AWS Well-Architected Agent AI Lens, Anthropic, akademischen Reviews):

Risiko-getierte Gates: nur konsequente Entscheidungen menschlich prüfen; „route none“ = unbounded autonomy, „route all“ = Rubber-Stamp-Approvals/Reviewer-Fatigue. [AWS](#)

➤ *Approve-before-act* (synchron) für irreversible/hochriskante Aktionen; *review-after-act* für niedrigriskante. *Schwellwert-basiert:* z.B. Rückerstattung unter Betrag X automatisch, darüber Approval; Aktion außerhalb Trainingsverteilung, über Budget, oder unter Confidence-Schwelle → eskalieren. [Arthur](#)

➤ *Maker-Checker* für höchststakige Aktionen (Agent schlägt vor, separater Mensch/Kontrolle genehmigt).

Safe-Default bei Timeout: wenn kein Approver reagiert → **deny**, nicht stillschweigend ausführen.

Alles auditieren: Reviewer-Identität, Zeitstempel, Aktion, Entscheidung, Eskalation. Hinweis: Die EU-AI-Act-Frist (August 2026) macht nachweisbare menschliche Aufsicht rechtlich verpflichtend. [Galileo AI](#)

➤

9. Architektur-Topologien (Muster A / B / C)

Muster A – Direkter nativer Plattform-MCP-Server. Die Zielplattform exponiert ihre Fähigkeiten selbst als MCP-Server; das LLM-Produkt verbindet sich direkt.

Vorteile: kein eigener Wartungsaufwand für den Server; Tools sind (idealerweise) agent-ergonomisch designt statt REST-1:1; Updates kommen vom Anbieter; native Server können direkt gegen Datenquellen gehen (kein Umweg über die eigene REST-API). [Mcpfabric](#)

➤

Nachteile: Kontrollverlust – man ist der Tool-Granularität, den Annotationen, dem Auth-Modell und der Versionierung des Anbieters ausgeliefert; kein Eingriff in Read/Write-Trennung oder Tool-Beschreibungen; Security-Exposure hängt von der Reife des Anbieter-Servers ab (viele öffentliche MCP-Server sind unreif – vgl. die Vorgänger-Recherche: 82% Path-Traversal-anfällig, 38–41% ganz ohne Auth).

Latenz: minimal (ein Hop weniger als bei Wrapper).

Wann: wenn der Anbieter einen produktionsreifen, sicherheitsgeprüften nativen Server hat, dessen Toolset gut zu den Use-Cases passt.

Muster B – Eigener vorgelagerter MCP-Server vor einer klassischen API. Selbstgebauter Adapter/Wrapper vor der REST/GraphQL-API (die Plattform hat *keinen* nativen MCP-Server).

Vorteile: volle Kontrolle über Tool-Granularität (Konsolidierung statt Endpoint-1:1), Beschreibungen, Annotationen, Read/Write-Trennung, Auth (Token-Exchange), Fehlermeldungen, Audit. Man kann exakt die agent-ergonomischen, aufgabenorientierten Tools bauen, die Anthropic empfiehlt.

Nachteile: Bau-, Test- und Betriebsaufwand; man kann nur exponieren, was die darunterliegende API hergibt (bei reinem 1:1-Wrapping bleiben die REST-Formen erhalten – der Mehrwert entsteht erst durch bewusstes Re-Design). OpenAPI-Generatoren (z.B. FastMCP `from_fastapi`, openapi-to-mcpserver) beschleunigen den Bau, erzeugen aber oft genau die unerwünschten 1:1-Wrapper.

Latenz: ein zusätzlicher Hop (Wrapper → API).

Wann: Standardfall, wenn kein (guter) nativer Server existiert, man Kontrolle über Sicherheit/Granularität braucht, oder mehrere Modelle/Agents denselben Zugang nutzen sollen. Faustregel (StudioBuildIt/Praxis 2026): „Default to MCP [eigener Server] unless you can name the reason not to.“ [StudioBuildIt](#)

↗

Muster C – Orchestrator/Gateway vor mehreren MCP-Servern. Zentrale Kontrollebene, die mehrere spezialisierte Server (Vector Search, Graph Search, strukturierte Daten ...) aggregiert und dem Host als einheitliche Schnittstelle präsentiert.

Vorteile: zentralisierte Auth (ein Endpoint, ein Auth-Integrationspunkt statt N), zentrales Audit/Observability, Rate-Limiting, Policy-Enforcement, Tool-Aggregation mit Namespacing/Disambiguierung, Tool-Filtering gegen Overload, Transport-Übersetzung (STDIO ↔ HTTP), permission-aware Tool-Katalog je Agent. Das Gateway ist der natürliche Ort für alles, was die Basis-Spec bewusst offenlässt (Audit, Rate-Limit, Approval).

Nachteile: zusätzliche Komponente (Betrieb, HA, Skalierung); das Gateway wird zum hochwertigen Angriffsziel und Single-Point-of-Trust – gerade hier ist Token-Passthrough besonders gefährlich (das Gateway hält Tokens aller Agents). Reifegrad heterogen, Feld noch jung.

Drei Sub-Patterns (Praxis): Reverse-Proxy (nur Routing, simpel), Aggregation (mehrere Server hinter einem Endpoint gemerged), Multi-Tenant (Tool-Zugriff isoliert je Team/Agent-Identität).

Latenz: zusätzlicher Hop; gute Gateways halten den Overhead klein (Bifrost wirbt mit ~11µs bei 5.000 req/s – Anbieterangabe).

Beispiele/Reifegrad: microsoft/mcp-gateway (Open Source, K8s-nativer Reverse-Proxy, session-aware, Entra-ID), IBM mcp-context-forge, Kong, Tyk, Solo.io agentgateway, Docker (Container-Isolation als Boundary), Microsoft Foundry (Azure API Management + Entra ID), Cloudflare Workers MCP, Envoy AI Gateway (1.500+ Stars, token-encoded Sessions, kein Redis nötig), WRITER MCP Gateway (interne Enterprise-Case-Study: automatische API-Ingestion + LLM-optimiertes Tool-Description-Rewriting gegen Kontext-Bloat). Ein Xenoss-Bericht nennt eine MCP-Adoption von 78% bei Produktions-KI-Teams und über 9.400 Server im öffentlichen Registry (Sekundärquelle).

Wie die Muster zusammenhängen: C ist orthogonal zu A/B – ein Gateway (C) kann *intern* sowohl native Anbieter-Server (A) als auch eigene Adapter (B) als Backends einbinden. In der Praxis ist die realistische Enterprise-Form eine *Mischung*: „two MCP servers + one custom REST endpoint“ hinter einer Kontrollebene ist die typische Gestalt von Produktions-Agents 2026. [StudioBuildIt](#)

↗

10. Zusätzliche Enterprise-Aspekte

Versionierung: Es gibt zwei Ebenen – (a) die *Protokoll-Revision* (datumsbasiert: 2024-11-05, 2025-03-26, 2025-06-18, 2025-11-25, 2026-07-28), im `initialize`-Handshake ausgehandelt; (b) *Tool-/Server-Versionierung*, für die es *keinen* Spec-Standard gibt. Best Practice: Tool-Versionen pinnen (Clients sollten nicht blind auto-updaten – Rug-Pull-Risiko), Upstream auf Patches beobachten und vor Deployment vetten. Semver für eigene Server; Breaking Changes an Tool-Schemata wie API-Breaking-Changes behandeln.

Rate Limiting: Kein Spec-Feature; auf Gateway-, App- und Tool-Ebene umsetzen (Defense-in-Depth, OWASP-Empfehlung). Wichtig: aus Gateway-Sicht ist *jede* MCP-Operation ein HTTP-POST – ein Limit „5/s“ erlaubt ~1 Tool-Call-Session/s, nicht 5 Calls (eine Session umfasst mehrere Requests). Daher in *Sessions*

denken und ggf. per-Tool-Limits via JSON-RPC-Method-Inspection setzen (Solo.io agentgateway). Multi-dimensionale Limits (Nutzer/Team/Tool-Kategorie/Zeitfenster) + Burst-Allowances gegen Runaway-Agent-Loops und Multi-Tenant-Contention (MintMCP). [Solo](#)

↗

Multi-Tenancy: Tenant-Isolation (Daten, Scopes, Rate-Limits) ist kein Spec-Feature; Muster: geteilter Server mit RLS (PostgreSQL Row-Level-Security)/tenant-scoped Tokens, für sensible Tenants dedizierte Instanzen/DBs. Reale Herausforderungen: pro-Tenant-OAuth-Lifecycles, vendor-spezifische Rate-Limits (z.B. NetSuite Concurrency-Slot-Modell statt Quota), harte Datenisolation. [Truto](#)

↗

Testing: Spec-Konformität testen (`tools/list` in CI, Annotationen validieren – sonst „safe-looking metadata“ auf riskanten Tools). Anthropic empfiehlt evaluationsgetriebene Entwicklung: Eval-Suite bauen, Tools iterativ (auch mit Claude selbst) gegen die Evals optimieren, lokalen MCP-Server in Claude Code/Desktop testen. [Modelcontextprotocol](#)

↗

Caching: Kein Spec-Standard. Resources eignen sich für cachebaren, kuratierten Read-Kontext; wiederholte Backend-Antworten cachen (Redis) senkt Latenz und Timeout-Risiko. `idempotentHint` hilft bei sicherem Retry/Caching.

Langlaufende Tool-Calls: Spec-Utilities `notifications/progress` (mit `progressToken` im `_meta` des Requests) für Fortschritt und `notifications/cancelled` für Abbruch. Für große Ergebnisse Pagination (opaker Cursor) statt Timeout-Erhöhung. `ping` als Keepalive. Cancellation-Propagation an Upstream-Server ist implementierungsabhängig und noch lückenhaft (vgl. IBM mcp-context-forge #1983).

Recommendations

Zielszenario: LLM-Chat-Produkt ↔ Backend-Plattform mit (1) strukturierten Daten, (2) Vector Search, (3) Graph Search.

Stufe 1 – Topologie festlegen (jetzt):

Baue einen **eigenen vorgelagerten MCP-Server (Muster B)** pro fachlicher Domäne, *nicht* API-1:1, sondern aufgabenorientiert konsolidiert. Konkret drei schlanke, klar abgegrenzte Tools statt Endpoint-Wrapping, z.B. `data_query` (strukturierte Daten, read-only), `semantic_search` (Vector), `graph_search`/`graph_expand` (Graph). Erwäge ein kombiniertes `hybrid_search` (Vector→Graph-Expansion), da genau dieses Muster (VectorGraphRAG) in der Literatur als überlegen gilt.

Setze **ein leichtes Gateway/Orchestrator (Muster C)** davor, sobald ≥2 Backend-Server existieren – als zentraler Auth-, Audit-, Rate-Limit- und Namespacing-Punkt. Start mit Reverse-Proxy/Aggregation; Multi-Tenant erst bei Bedarf.

Nutze **Muster A (nativer Server)** nur, wo die Plattform selbst einen *produktionsreifen, sicherheitsgeprüften* nativen MCP-Server bietet, dessen Toolset passt – und binde ihn dann *als Backend hinter dein Gateway* ein (nicht ungefiltert direkt an den Host).

Stufe 2 – Sicherheit & Auth (vor Produktion):

OAuth 2.1: MCP-Server als reiner Resource Server, externer Authorization Server (Entra ID/Okta/Keycloak), PKCE + Resource Indicators (RFC 8707) verpflichtend.

Kein Token-Passthrough. Endnutzer-Identität via **Token-Exchange (RFC 8693)** im Gateway/Server gegen scoped Backend-Tokens tauschen. Delegierte Nutzeridentität statt geteiltem Service-Account.

Tool-Ebene: RBAC/ABAC + Least-Privilege-Scopes; read-Tools und write-Tools mit *getrennten* Scopes.

Stufe 3 – Read/Write & Approval:

Read-Tools (`[readOnlyHint:true]`): Auto-Approve, aggressives Caching, breiter Zugang. Die drei Search-/Query-Tools sind read-only.

Write-/destruktive Tools: **getrennter Scope + Human-Approval-Gate** (approve-before-act), risikogetriert mit Safe-Default „deny“ bei Timeout, Maker-Checker für Höchststakiges. Enforcement deterministisch im Gateway/Server, *nicht* auf Annotationen verlassen (Annotationen zusätzlich für UX setzen).

Stufe 4 – Tool-Granularität & Discovery:

Aktive Toolzahl je Agent klein halten (Ziel-Größenordnung ~5–15; Tool-Definitionen < ~40% Kontext).

Search-fokussiert statt list-all, semantische Rückgaben statt UUIDs, `[concise]`/`[detailed]`-Response-Parameter, Response-Cap (z.B. 25k Token).

Bei Wachstum über ~30–40 Tools: retrieval-basierte Tool-Auswahl (RAG-MCP) oder Code-Execution/Progressive-Disclosure evaluieren (beide noch jung – zuerst gegen eigene Evals messen). Namespacing per Präfix (`[data_]`, `[vector_]`, `[graph_]`) von Beginn an.

Stufe 5 – Betrieb & Compliance:

Audit-Trail im Gateway: principal (Nutzer+Agent), Tool, Server, Argumente, Ergebnis, Latenz, Approval, Zeitstempel; unveränderlich, SIEM-exportierbar, Content-Logging pro Umgebung abschaltbar (PII).

Rate-Limits in Sessions denken, multi-dimensional (Nutzer/Team/Tool), Burst-Allowance gegen Runaway-Loops.

Tool-Versionen pinnen, Annotation-Drift überwachen, `[tools/list]`-Konformität in CI testen.

Langläufer: Progress-Notifications + Pagination + Timeouts statt Timeout-Erhöhung.

Schwellwerte, die die Empfehlung ändern würden:

Nur ein Backend-Server, ein Consumer, kein Multi-Tenant, niedriges Compliance-Niveau: Gateway (C) ist Overkill
→ Muster B direkt an den Host, Audit im Server. Gateway einführen, sobald ein zweiter Server/Consumer oder regulatorische Anforderungen dazukommen.

Plattform liefert reifen nativen Server (A): eigenen Wrapper (B) einsparen – aber nur, wenn Tool-Granularität, Auth-Modell und Sicherheitsreife die Anforderungen erfüllen; sonst B beibehalten.

Tool-Auswahl-Genauigkeit fällt in Evals unter Zielwert (z.B. Auswahl-Accuracy sinkt spürbar / >30–40 aktive Tools): Tool-Filtering/RAG-MCP aktivieren, Tools weiter konsolidieren.

Caveats

Reifegrad zweigeteilt: Auth (OAuth 2.1), Tool-Annotationen, Elicitation/Sampling, Progress/Cancellation sind *offizieller Spec-Standard*. Audit-Logging, Token-Exchange zum Backend, Tool-Discovery-at-Scale, Human-Approval, Rate-Limiting, Multi-Tenancy und Versionierung sind *nicht* im Kern-Protokoll und müssen selbst gebaut oder zugekauft werden.

Annotationen sind nur Hints, keine Garantien – niemals als Sicherheitskontrolle verwenden.

Gateway-Anbieterzahlen kritisch behandeln: Angaben wie „98% Token-Reduktion“, „11µs Overhead“, „78% Adoption“, „88% hatten Incidents“ stammen von interessierten Anbietern/Surveys und sind indikativ, nicht unabhängig validiert. Anthropic's 98,7%-Token-Ersparnis (150.000 → 2.000 Token) ist ein *illustratives Einzelbeispiel*, kein neutraler Benchmark.

RAG-MCP / Code-Execution / Progressive Disclosure sind vielversprechend, aber jung; ihre Erfolgszahlen stammen von den jeweiligen Autoren. Vor breitem Einsatz gegen eigene Evaluations messen.

Client-Heterogenität: Elicitation-, Sampling- und Namespacing-Verhalten unterscheiden sich je Client erheblich; „list_changed“/dynamische Discovery und Cancellation-Propagation sind teils lückenhaft implementiert.

Spec bewegt sich schnell: Datumsbasierte Revisionen ändern sich häufig (zuletzt Release-Candidate 2026-07-28); konkrete MUST/SHOULD-Formulierungen gegen die zum Implementierungszeitpunkt gültige Revision verifizieren.

Die in der Vorgänger-Recherche dokumentierten Sicherheitsbefunde (CVE-2025-6514, Tool Poisoning, 82% Path-Traversal-anfällig, 38–41% ohne Auth) unterstreichen: die *Reife öffentlicher MCP-Server ist niedrig* – ein Grund, native Server (Muster A) nur geprüft und hinter einem Gateway einzusetzen.